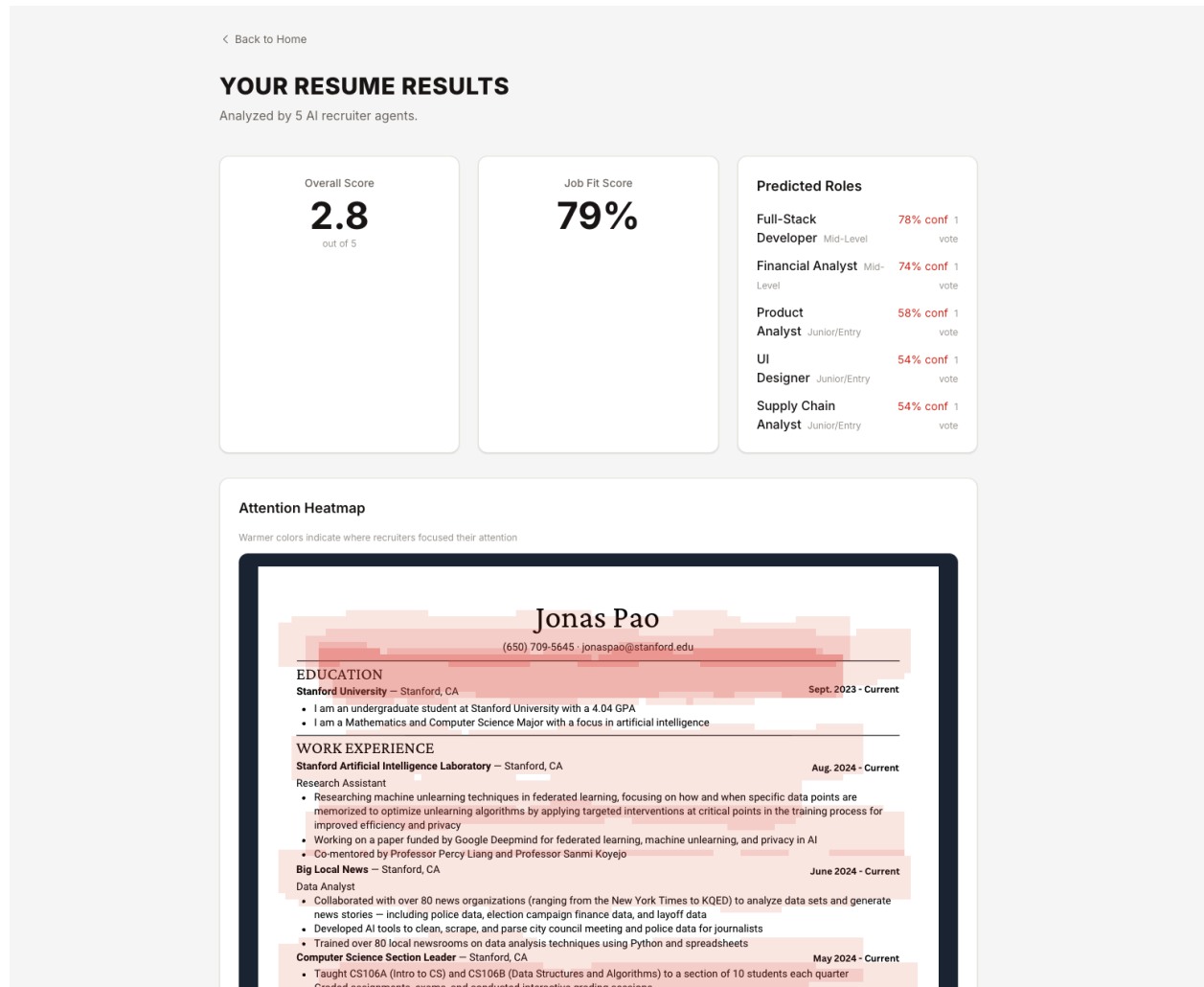




Example above. My plan is to do a live demo walkthrough.

## # How ResumeScope Works

### ## System Overview

ResumeScope is a resume review platform with two user roles:

- **Students** upload a resume and receive automated feedback from a panel of AI "recruiter" agents.
- **Labelers** (real recruiters) review resumes manually — painting brush strokes over areas they focus on, rating the resume, and writing feedback. Their behavior is stored as training data.

The core question the system answers: **"Where will a recruiter look on this resume, and what will they think?"**

---

## ## 1. Training: How Recruiter Behavior Gets Logged

### ### Labeler Session Setup

When a recruiter visits the labeler flow, they first create a profile (`POST /api/labelers/session`) with:

- **Job function** (e.g. Engineering, Finance, Design)
- **Industry** (e.g. Tech, Healthcare, Consulting)
- **Level** (e.g. Senior, Manager, Junior/Entry)
- **"What do you look for first?"** — free text describing their review priorities

This creates a `labeler\_sessions` record in the database.

### ### Calibration

Before reviewing real resumes, the labeler must pass a calibration step. A static PDF shows 5 horizontal reference lines at known Y-positions ([0.13, 0.25, 0.44, 0.63, 0.82]). The labeler paints over each line with the brush tool. The system verifies coverage within a  $\pm 0.03$  tolerance in normalized Y coordinates. This ensures the brush input is consistent and usable.

### ### Resume Review (Data Capture)

For each resume the labeler reviews, the system records:

| Data Point            | Format                                                      | Purpose                                          |
|-----------------------|-------------------------------------------------------------|--------------------------------------------------|
| Brush strokes         | Array of `{pageIndex, points: [{x, y}], brushWidth, order}` | Spatial attention — where the recruiter looked   |
| Overall rating        | 1-5 integer                                                 | Quality judgment                                 |
| Strengths             | Free text                                                   | Positive feedback                                |
| Weaknesses            | Free text                                                   | Negative feedback                                |
| Guessed role & level  | Strings                                                     | What job the recruiter thinks the candidate fits |
| Recommended companies | Free text                                                   | Optional company suggestions                     |
| Time spent            | Seconds                                                     | How long the review took                         |

All brush coordinates are **normalized to 0.0–1.0** (fraction of page width/height), making them resolution-independent. This data is stored in the `reviews` table with brush strokes as JSONB in the `brush\_highlights` column.

### ### What "Brush Strokes" Represent

The brush system simulates eye-tracking. Instead of hardware gaze data, recruiters manually paint over the parts of a resume they focus on. Each stroke is a polyline of 15-30 points with a 35px-wide semi-transparent yellow brush. The order field records the sequence — which areas the recruiter looked at first, second, etc.

---

## ## 2. Inference: How the Model Predicts Recruiter Behavior

### ### Agent Personas

The system maintains a set of **agent personas** — synthetic recruiter profiles stored in the ``agent_personas`` table. Each persona has a name, job function, industry, level, and a description of what they look for first. Eight are pre-seeded:

| Persona                 | Job Function | Level        |
|-------------------------|--------------|--------------|
| Senior Tech Recruiter   | Engineering  | Senior       |
| Finance Hiring Manager  | Finance      | Manager      |
| Product Design Lead     | Design       | Senior       |
| Startup Product Manager | Product      | Mid-Level    |
| Healthcare Ops Director | Operations   | Director     |
| Data Science Recruiter  | Data Science | Mid-Level    |
| Consulting Partner      | Consulting   | VP+          |
| Junior HR Coordinator   | HR           | Junior/Entry |

### ### Ensemble Selection

When a resume is uploaded, the system selects up to **5 personas** to form a review panel:

1. Personas whose ``job_function`` matches the resume's target job get priority.
2. Remaining slots are filled with diverse personas (different job functions).
3. If there aren't enough, any active persona fills the remaining slots.

### ### Two Inference Modes

#### #### Fake Runner (Deterministic, Template-Based)

**File:** ``backend/services/agent_runner_fake.py``

The fake runner uses no ML model. It generates plausible-looking results from templates and weighted random sampling, seeded deterministically with ``hash(resume_id + persona_id)``. This means the same resume + persona pair always produces the same output.

What it generates:

- **Simulated brush strokes** (3-6 per resume): Placed probabilistically across typical resume sections. The Experience section (Y: 0.18–0.55) has 90% selection weight; Header (Y: 0.05–0.10) only 40%. Each stroke is a horizontal polyline with slight vertical jitter to look hand-drawn.
- **Guessed roles**: 1-3 roles sampled from a job-function-specific lookup table (e.g., Engineering maps to Software Engineer, Backend Developer, etc.).
- **Fit score**: `random.uniform(45, 92)` — a 0-100 percentage.
- **Overall score**: `random.uniform(2.0, 4.5)` — a 1-5 rating.
- **Strengths/weaknesses/tips**: Randomly selected from predefined template lists, parameterized by job function and target job title.

The fake runner is fast (<100ms), free, and reliable. It's the default for demos and testing.

#### #### Live Runner (LLM-Based)

**File:** `backend/services/agent_runner_live.py`

The live runner calls **Gemini 2.0 Flash** via the OpenRouter API. It constructs a two-part prompt:

**System prompt** (from `agent_persona.txt`):

...

You are {name}, a {level} professional in {job\_function} within the {industry} industry.  
When reviewing resumes, the first thing you look for is: {what\_look\_for\_first}  
Apply your professional perspective and hiring experience to evaluate the resume.

...

**User prompt** (from `agent_review.txt`):

The full resume text plus instructions to return structured JSON with scores, roles, strengths, weaknesses, tips, and highlight spans.

The LLM returns a JSON object with the same fields as the fake runner. If the API call fails for any reason, the system **automatically falls back** to the fake runner and marks the result as `run_mode: "live_fallback"`.

#### #### Mixed Mode

In `mixed` mode, the first 2 agents use the live runner and the rest use fake. This balances cost and quality.

#### ### How Training Data Connects to Prediction

The current system does **not** retrain a model on labeler data automatically. Instead:

1. **Labeler reviews** populate the `reviews` table with real recruiter brush patterns, ratings, and feedback.
2. **Agent personas** are pre-seeded but can be created from labeler sessions (the `source\_labeler\_session\_id` foreign key exists for this purpose).
3. The **fake runner's section probability weights** (Experience: 90%, Education: 70%, Skills: 60%, etc.) are hand-tuned heuristics that approximate real recruiter attention patterns.
4. The **live runner** relies on the LLM's general knowledge of recruiting behavior, shaped by the persona prompt.

The labeler data collection pipeline is designed so that a future iteration could:

- Analyze real brush stroke distributions to update the fake runner's section weights.
- Create new agent personas derived from individual labeler profiles.
- Fine-tune or few-shot prompt the LLM with real recruiter examples.

---

### ## 3. Aggregation: Combining Agent Results

**File:** `backend/services/aggregator.py`

After all agents finish reviewing a resume, the aggregator combines their outputs into a single `aggregated\_results` record.

### ### Spatial Heatmap Construction

This is the most technically involved step. It converts individual brush strokes from all agents into a unified attention heatmap:

1. **Grid creation**: A 100-wide x 140-tall grid is created per page (14,000 cells total).
2. **Per-agent rasterization**: For each agent's strokes, a binary grid tracks which cells were "hit." For each line segment between consecutive brush points:
  - The brush radius is normalized to grid units:  $(\text{brushWidth} / 612) * \text{GRID\_WIDTH} / 2$  (612 is a standard PDF point width).
  - The algorithm steps along the segment with 2x oversampling.
  - Every grid cell within the brush radius is marked as hit.
3. **Accumulation**: Per-agent binary grids are summed across all agents.
4. **Normalization**: Each cell's count is divided by `num\_agents`, yielding a 0.0–1.0 intensity (0 = no agent looked here, 1.0 = every agent looked here).

The resulting `PageHeatmap` is rendered on the frontend as a red-tinted overlay on the PDF.

### ### Section-Level Attention

The aggregator also detects resume sections from the extracted text using regex patterns (e.g., "experience", "education", "skills", "projects"). Each section's character range is mapped to a Y-band on the heatmap grid, and the average cell intensity within that band becomes the section's attention score. This powers the "Section Attention" bar chart.

#### ### Role Voting

All `guessed_roles` from all agents are grouped by `(role, level)`. The aggregator counts votes and averages confidence scores, then returns the top 5 roles sorted by vote count.

#### ### Score Averaging

- `avg_fit_score`: Mean of all agents' fit scores (0-100 scale).
- `avg_overall_score`: Mean of all agents' overall scores (1-5 scale).

#### ### Text Deduplication

Strengths, weaknesses, takeaways, and tips from all agents are merged into consensus lists. A substring containment check deduplicates entries — if one agent's feedback is a substring of another's, the longer version is kept.

---

## ## 4. Display: What the Student Sees

The results page (`frontend/src/pages/student/ResultsPage.tsx`) renders:

| Component                | Data Source                                                                                                                       | What It Shows                                                                      |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Overall Score Card       | <code>avg_overall_score</code>                                                                                                    | X.X out of 5                                                                       |
| Job Fit Score Card       | <code>avg_fit_score</code>                                                                                                        | XX%                                                                                |
| Predicted Roles          | <code>top_roles</code>                                                                                                            | Role, level, confidence %, vote count                                              |
| Attention Heatmap        | <code>spatial_heatmap</code>                                                                                                      | Red overlay on the PDF — warmer = more recruiter focus                             |
| Section Attention        | <code>section_scores</code>                                                                                                       | Horizontal bar chart per resume section                                            |
| Key Takeaways            | <code>consensus_takeaways</code>                                                                                                  | Bullet list                                                                        |
| Detailed Feedback        | <code>consensus_strengths</code> , <code>consensus_weaknesses</code> , <code>general_tips</code> , <code>job_specific_tips</code> | Categorized lists                                                                  |
| Individual Agent Reviews | <code>agent_runs</code>                                                                                                           | Expandable cards per persona with their individual scores, reasoning, and feedback |

The standalone `AgentCard` component (`frontend/src/features/AgentCard.tsx`) uses color-coded score badges:

- **Red** (score < 2.5): Poor
- **Amber** (score 2.5–3.5): Moderate
- **Green** (score > 3.5): Good

---

## ## 5. Data Flow Summary

...

Student uploads PDF

|

v

Text extracted (pdfplumber) + PDF stored (Supabase Storage)

|

v

Ensemble of up to 5 agent personas selected

|

v

Each agent reviews resume independently:

- Fake: deterministic templates + weighted random sampling
- Live: Gemini 2.0 Flash via OpenRouter API
- Mixed: first 2 live, rest fake

|

v

Individual results stored in agent\_runs table

|

v

Aggregator combines all results:

- Rasterize brush strokes into 100x140 heatmap grid
- Detect sections, compute per-section attention
- Vote on roles, average scores
- Deduplicate text feedback

|

v

Aggregated results stored, resume status set to 'completed'

|

v

Frontend polls status, fetches results, renders dashboard

...

---

## ## 6. Tech Stack

| Layer | Technology |  
|---|---|  
| Backend | FastAPI (Python 3.12+) |  
| Frontend | React + Vite + TypeScript + Tailwind CSS |  
| Database | Supabase (PostgreSQL) |  
| File Storage | Supabase Storage |  
| LLM | Gemini 2.0 Flash via OpenRouter |  
| PDF Parsing | pdfplumber (Python) |  
| PDF Rendering | react-pdf (Frontend) |